Elements of AIML
Unit 02 – Lecture 03 Notes
Clause Form, Resolution & Unification

Tofik Ali

March 28, 2026

Contents

1 Lecture Overview **2**

2 How to use these notes (self-study) **2**

3 Learning Outcomes **2**

4 Before you start **2**

5 Keywords **3**

6 Abbreviations and symbols **3**

7 Refutation view of entailment **3**

8 Clause form (CNF) in propositional logic **4**
8.1 What exactly is a clause set? 4
8.2 CNF conversion recipe (PL) 4
8.3 Mini example 4
8.4 Worked example (distribution is required) 4
8.5 Worked example (biconditional) 5
8.6 Useful CNF simplifications (optional but practical) 5

9 Resolution in propositional logic **5**
9.1 Resolution rule 5
9.2 Why resolution is useful 5
9.3 Worked proof: $(p \to q), p \models q$ 5
9.4 Worked proof: $(p \to q), (q \to r), p \models r$ 6
9.5 Example: detecting inconsistency (unsatisfiable KB) 6
9.6 Algorithmic view (how an automated prover uses it) 6

10 Resolution in first-order logic (high-level) **6**
10.1 Clause form conversion in FOL (more detail) 7
10.2 Worked clause form example (Skolemization) 7
10.3 Worked end-to-end example: "Socrates is mortal" 7

11 Unification **8**
11.1 Definition 8

11.2 Substitutions (what θ really is) . . . 8
11.3 Most general unifier (MGU) . . . 8
11.4 Unification algorithm (high-level idea) . . . 9
11.5 Examples . . . 9
11.6 Occurs-check intuition . . . 9

12 Practice (with answers) 9

13 Exit questions (with answers) 10

14 Common pitfalls (and how to avoid them) 10

15 Summary 11

1 Lecture Overview

This lecture introduces **resolution**, a widely used inference method for automated reasoning, and the supporting ideas:

- converting sentences to **clause form** (CNF),
- proving entailment by **refutation** ($KB \wedge \neg\alpha$ unsatisfiable),
- **unification** to extend resolution from propositional logic to first-order logic.

Sources: Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 7–9 (Logical Agents, First-Order Logic, Inference).;Huth and Ryan, *Logic in Computer Science* (2e), selected sections on propositional and first-order logic.;Genesereth and Nilsson, *Logical Foundations of Artificial Intelligence*, selected chapters.

2 How to use these notes (self-study)

Resolution becomes easy only after you are comfortable with two routines: (1) converting sentences to CNF (clause sets), and (2) performing resolution steps carefully until no new clauses appear. For FOL resolution, add a third routine: unification (computing substitutions). While studying, always keep a paper "clause set" and write each new resolvent explicitly.

3 Learning Outcomes

- Convert simple propositional statements to CNF.
- Use resolution to show entailment in propositional logic for small KBs.
- Explain the main extra steps needed for FOL resolution.
- Define unification and compute simple MGUs.

4 Before you start

This lecture assumes two earlier routines are already familiar:

- from **Unit 02 Lecture 01**: entailment, unsatisfiability, and basic CNF conversion,
- from **Unit 02 Lecture 02**: predicates, quantifiers, and substitution.

If any of those ideas feel weak, quickly revisit the earlier notes first. Resolution is mechanical only after the prerequisite language is stable.

5 Keywords

- **CNF:** conjunctive normal form (AND of OR clauses).
- **Literal:** an atom or its negation.
- **Clause:** a disjunction of literals.
- **Resolution:** rule of inference on clauses (refutation proof method).
- **Refutation:** prove $KB \models \alpha$ by showing $KB \wedge \neg\alpha$ is unsatisfiable.
- **Unification:** find a substitution to make expressions identical.
- **MGU:** most general unifier (least restrictive substitution).

6 Abbreviations and symbols

- **PL**: propositional logic.
- **FOL**: first-order logic (predicate logic).
- KB: knowledge base (set of sentences/facts/rules).
- α: query/goal sentence.
- $\models$: semantic entailment (truth in *all* models).
- $\vdash$: syntactic derivation/provability in a proof system.
- $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$: NOT, AND, OR, IMPLIES, IFF.
- **CNF**: conjunctive normal form; **DNF**: disjunctive normal form.
- $\square$: empty clause (contradiction).
- θ: a substitution (mapping variables $\rightarrow$ terms); **MGU**: most general unifier.
- **Skolem constant/function**: new symbols introduced to eliminate $\exists$ in clause form.

7 Refutation view of entailment

In logic for AI we distinguish:

- **Semantics (meaning):** what is true in the world(s) described by our sentences.
- **Syntax (proof):** how we can mechanically derive new sentences from existing ones.

Two important relations:

- **Entailment** $KB \models \alpha$: in *every* model where all sentences in KB are true, α is also true.
- **Derivation** $KB \vdash \alpha$: α can be proven from KB using a chosen proof procedure (rules of inference).

If a proof system is **sound**, then $KB \vdash \alpha \Rightarrow KB \models \alpha$ (no false conclusions). If it is **complete**, then $KB \models \alpha \Rightarrow KB \vdash \alpha$ (it can prove everything that is true).

The central equivalence used by resolution:

$$KB \models \alpha \quad \Leftrightarrow \quad KB \wedge \neg\alpha \text{ is unsatisfiable}$$

So, to prove entailment:

1. Take all sentences in KB and the negation of the query $\neg\alpha$.
2. Convert them into CNF (set of clauses).
3. Apply resolution repeatedly.
4. If you derive the **empty clause** ($\square$), a contradiction is reached, so $KB \models \alpha$.

8 Clause form (CNF) in propositional logic

8.1 What exactly is a clause set?

In resolution we usually treat CNF as a **set of clauses**:

$$(C_1) \wedge (C_2) \wedge \cdots \wedge (C_m)$$

where each C_i is a disjunction of literals. For automated reasoning it is convenient to store CNF as:

$$\{C_1, C_2, \ldots, C_m\}$$

i.e., a *set* (or list) of clauses.

8.2 CNF conversion recipe (PL)

1. Eliminate $\leftrightarrow$ and $\rightarrow$.
2. Push negations inward (De Morgan + double negation).
3. Distribute $\vee$ over $\wedge$ to obtain an AND of ORs.
4. Flatten nested AND/OR groups.

8.3 Mini example

Convert $(p \rightarrow q) \wedge p$:

1. $(\neg p \vee q) \wedge p$
2. CNF clauses: $(\neg p \vee q)$ and (p)

8.4 Worked example (distribution is required)

Convert $p \rightarrow (q \wedge r)$:

1. Eliminate implication:
$$p \rightarrow (q \wedge r) \quad \equiv \quad \neg p \vee (q \wedge r)$$
2. Distribute $\vee$ over $\wedge$:
$$\neg p \vee (q \wedge r) \quad \equiv \quad (\neg p \vee q) \wedge (\neg p \vee r)$$
3. Clauses: $(\neg p \vee q)$ and $(\neg p \vee r)$.

8.5 Worked example (biconditional)

Convert $p \leftrightarrow q$:

1. Replace IFF:
$$(p \leftrightarrow q) \equiv (p \to q) \wedge (q \to p)$$

2. Eliminate implications:
$$(\neg p \vee q) \wedge (\neg q \vee p)$$

3. Already in CNF (two clauses).

8.6 Useful CNF simplifications (optional but practical)

After conversion we often simplify the clause set:

- Remove duplicate literals inside a clause: $(p \vee p \vee q) \Rightarrow (p \vee q)$.
- Remove clauses that are always true (tautological): $(p \vee \neg p \vee q)$ can be dropped.
- Use **unit clauses** like (p) effectively (unit resolution).

9 Resolution in propositional logic

9.1 Resolution rule

Given two clauses $(A \vee p)$ and $(B \vee \neg p)$, resolve on p to infer $(A \vee B)$.

9.2 Why resolution is useful

- It is **sound**: any resolvent derived is logically implied by the parents.
- For propositional logic, it is also **complete** for refutation: if the clause set is unsatisfiable, resolution can derive $\square$.
- It reduces reasoning to a small number of mechanical steps, which is why SAT solvers and automated provers are built around similar ideas.

9.3 Worked proof: $(p \to q), p \models q$

Let $KB = \{p \to q,\ p\}$ and query $\alpha = q$.

1. Add $\neg q$ (negated query).

2. Convert to CNF clauses:
$$(\neg p \vee q),\ (p),\ (\neg q)$$

3. Resolve $(\neg p \vee q)$ with (p) to get (q).

4. Resolve (q) with $(\neg q)$ to get $\square$ (empty clause).

Hence $KB \models q$.

9.4 Worked proof: $(p \to q), (q \to r), p \models r$

Let $KB = \{p \to q,\ q \to r,\ p\}$ and query $\alpha = r$.

1. Add $\neg r$.
2. CNF clauses:
$$C_1 : (\neg p \vee q), \quad C_2 : (\neg q \vee r), \quad C_3 : (p), \quad C_4 : (\neg r)$$
3. Resolve C_1 with C_3 on p to get $C_5 : (q)$.
4. Resolve C_2 with C_5 on q to get $C_6 : (r)$.
5. Resolve C_6 with C_4 on r to derive $\square$.

Therefore $KB \models r$.

9.5 Example: detecting inconsistency (unsatisfiable KB)

Suppose we have:
$$(p \vee q), \quad (\neg p), \quad (\neg q)$$
Resolve $(p \vee q)$ with $(\neg p)$ to get (q), then resolve (q) with $(\neg q)$ to get $\square$. So the set is **unsatisfiable** (inconsistent).

9.6 Algorithmic view (how an automated prover uses it)

An automated prover typically:

- maintains a growing set of clauses,
- repeatedly chooses pairs of clauses with complementary literals,
- adds resolvents until either $\square$ is derived or no new clauses can be produced.

In practice, **search strategy** matters (unit preference, set-of-support, subsumption), but the core logical rule stays the same.

10 Resolution in first-order logic (high-level)

FOL resolution is similar, but clause form conversion can include additional steps (presented at a conceptual level here):

- remove implications,
- move negations inward,
- standardize variables (avoid clashes),
- handle quantifiers (often by Skolemization for $\exists$),
- drop universal quantifiers (implicit in clauses),
- produce a set of clauses.

Then resolution uses unification to match literals.

10.1 Clause form conversion in FOL (more detail)

To convert a FOL sentence to clause form for resolution, a common pipeline is:

1. Eliminate $\leftrightarrow$ and $\rightarrow$.
2. Push $\neg$ inward so that it applies only to atomic predicates.
3. Rename variables so that different sentences do not accidentally share variable names (**standardize apart**).
4. Move quantifiers to the front (prenex form) if needed.
5. Eliminate $\exists$ using **Skolemization**:
   - if $\exists y$ is inside the scope of $\forall x$, replace y by a new function $f(x)$;
   - if $\exists y$ has no surrounding $\forall$, replace y by a new constant k.
6. Drop $\forall$ (universals are implicit in clause variables).
7. Distribute $\vee$ over $\wedge$, then split into clauses.

10.2 Worked clause form example (Skolemization)

Sentence:

$$\forall x\ (Student(x) \rightarrow \exists y\ Enrolled(x, y)) \quad \text{“Every student is enrolled in some course.”}$$

Steps:

1. Remove implication:
$$\forall x\ (\neg Student(x) \vee \exists y\ Enrolled(x, y))$$
2. Pull $\exists$ out (since y does not appear in $\neg Student(x)$):
$$\forall x\ \exists y\ (\neg Student(x) \vee Enrolled(x, y))$$
3. Skolemize y with a new function $f(x)$:
$$\forall x\ (\neg Student(x) \vee Enrolled(x, f(x)))$$
4. Drop $\forall$ and read as one clause:
$$\neg Student(x) \vee Enrolled(x, f(x))$$

The Skolem function $f(x)$ should be read as “the course chosen for x” (it may differ for different x).

10.3 Worked end-to-end example: “Socrates is mortal”

English statements:

- Every human is mortal.
- Socrates is a human.
- Query: Is Socrates mortal?

Step 1: translate to FOL.

$$\forall x\ (Human(x) \to Mortal(x)), \qquad Human(Socrates)$$

Query:

$$Mortal(Socrates)$$

Step 2: set up refutation. To prove $KB \models Mortal(Socrates)$, add the negated query:

$$\neg Mortal(Socrates)$$

Step 3: convert to clause form.

- $\forall x\ (Human(x) \to Mortal(x))$ becomes $\neg Human(x) \vee Mortal(x)$.
- $Human(Socrates)$ is already an atomic clause.
- $\neg Mortal(Socrates)$ is already an atomic clause.

Clause set:

$$C_1 : (\neg Human(x) \vee Mortal(x)), \quad C_2 : Human(Socrates), \quad C_3 : \neg Mortal(Socrates)$$

Step 4: resolve with unification. Resolve C_1 with C_2 using substitution $\{x/Socrates\}$:

$$C_4 : Mortal(Socrates)$$

Now resolve C_4 with C_3:

$$\square$$

Because the empty clause is derived, the negated query is inconsistent with the KB, so the original query is entailed.

11 Unification

11.1 Definition

Unification finds a substitution θ such that $E_1\theta = E_2\theta$.

11.2 Substitutions (what θ really is)

A substitution θ is a finite set of bindings like:

$$\theta = \{x/t,\ y/u,\ z/v\}$$

meaning "replace x by term t, y by term u, etc.". Applying θ to an expression is written $E\theta$.

11.3 Most general unifier (MGU)

There may be many unifiers; we prefer the **most general** one:

- θ is more general than σ if σ can be obtained by adding extra substitutions after θ.
- MGU keeps as many variables free as possible (least restrictive).

11.4 Unification algorithm (high-level idea)

Unification can be seen as repeatedly simplifying a set of equations. One simple conceptual view:

```
UNIFY(E1, E2):
  if E1 == E2: return {}
  if E1 is a variable x: return {x / E2} (if occurs-check passes)
  if E2 is a variable x: return {x / E1} (if occurs-check passes)
  if E1 = f(a1,...,an) and E2 = f(b1,...,bn):
      unify each pair (ai, bi) and combine substitutions
  else: fail
```

In real provers, the implementation is more careful, but the cases above explain the intuition.

11.5 Examples

- Unify $Likes(x, AI)$ with $Likes(Ali, AI)$:

$$\theta = \{x/Ali\}$$

- Unify $Parent(x, y)$ with $Parent(Ali, Sara)$:

$$\theta = \{x/Ali,\ y/Sara\}$$

- Unify $Knows(x, f(y), y)$ with $Knows(Ali, f(z), z)$:

$$\theta = \{x/Ali,\ y/z\}$$

11.6 Occurs-check intuition

Unifying x with $f(x)$ would require an infinite structure. So a correct unification algorithm rejects such cases (the occurs-check).

12 Practice (with answers)

P1. Convert to CNF: $(p \to q) \land (q \to r)$.

Answer: $(\neg p \lor q) \land (\neg q \lor r)$.

P2. Show by resolution: $(p \to q), (q \to r), p \models r$.

Answer (details): Add $\neg r$. CNF clauses: $(\neg p \lor q)$, $(\neg q \lor r)$, (p), $(\neg r)$. Resolve $(\neg p \lor q)$ with (p) to get (q). Resolve $(\neg q \lor r)$ with (q) to get (r). Resolve (r) with $(\neg r)$ to get $\square$.

P3. Find an MGU for $Knows(x, f(y))$ and $Knows(Ali, f(z))$.

Answer: $\{x/Ali,\ y/z\}$ (or equivalently $\{x/Ali,\ z/y\}$ depending on variable choice).

P4. Convert to CNF: $p \to (q \land r)$.

Answer: $\neg p \lor (q \land r) \equiv (\neg p \lor q) \land (\neg p \lor r)$.

P5. Convert to clause form: $\forall x\ (P(x) \to \exists y\ Q(x, y))$.

Answer: $\forall x\ (\neg P(x) \lor \exists y\ Q(x, y)) \equiv \forall x \exists y (\neg P(x) \lor Q(x, y))$. Skolemize y as $f(x)$: $\forall x (\neg P(x) \lor Q(x, f(x)))$. Drop $\forall$: clause $(\neg P(x) \lor Q(x, f(x)))$.

P6. Give a small inconsistent clause set and show $\square$ by resolution.

Answer: $(p \vee q)$, $(\neg p)$, $(\neg q)$. Resolve $(p \vee q)$ with $(\neg p)$ to get (q), then resolve (q) with $(\neg q)$ to get $\square$.

13 Exit questions (with answers)

Q0. Entailment vs derivation?

Answer: $KB \models \alpha$ is a semantic claim about truth in all models; $KB \vdash \alpha$ means there is a proof of α from KB in a chosen proof system.

Q1. Why do we convert to CNF before resolution?

Answer: the resolution rule operates on clauses (disjunctions of literals). CNF provides a standard clause representation enabling systematic inference.

Q2. State the resolution rule for propositional logic.

Answer: from $(A \vee p)$ and $(B \vee \neg p)$ infer $(A \vee B)$.

Q3. Explain unification with an example.

Answer: unification finds a substitution to make expressions identical; e.g., unify $Likes(x, AI)$ with $Likes(Ali, AI)$ using $\{x/Ali\}$.

Q4. Why does the occurs-check matter?

Answer: it prevents invalid substitutions like $x = f(x)$ that would imply infinite terms and break correctness.

Q5. What does the empty clause $\square$ mean in resolution?

Answer: it represents an explicit contradiction: there is no possible truth assignment/model that can satisfy all derived clauses, so the clause set is unsatisfiable.

Q6. Why is Skolemization needed in FOL clause form?

Answer: resolution works on universally quantified clauses. Skolemization eliminates existential quantifiers by introducing new constants/functions that preserve satisfiability, allowing us to represent "there exists" choices inside clauses.

14 Common pitfalls (and how to avoid them)

- **Forgetting the refutation setup:** to prove $KB \models \alpha$, you must add $\neg\alpha$ and try to derive $\square$.
- **CNF conversion mistakes:** especially distributing $\vee$ over $\wedge$ and pushing negations inward correctly.
- **Variable capture/clashes in FOL:** standardize variables apart before Skolemization and clause extraction.
- **Treating Skolem symbols as "real" names:** they are placeholders introduced to remove $\exists$ while preserving satisfiability.

- **Unification errors:** apply substitutions consistently and remember the occurs-check intuition (x cannot unify with $f(x)$).

15 Summary

Resolution provides an automated reasoning method by reducing entailment to unsatisfiability, using CNF clauses and (for FOL) unification.

Sources: Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 7–9 (Logical Agents, First-Order Logic, Inference).;Huth and Ryan, *Logic in Computer Science* (2e), selected sections on propositional and first-order logic.;Genesereth and Nilsson, *Logical Foundations of Artificial Intelligence*, selected chapters.